SIWES PRESENTATION

MFB Core System

A double-entry accounting and lending core for a microfinance bank.

PRESENTED BY
[TO FILL: Full name]

MATRIC NUMBER
[TO FILL: Matric no.]

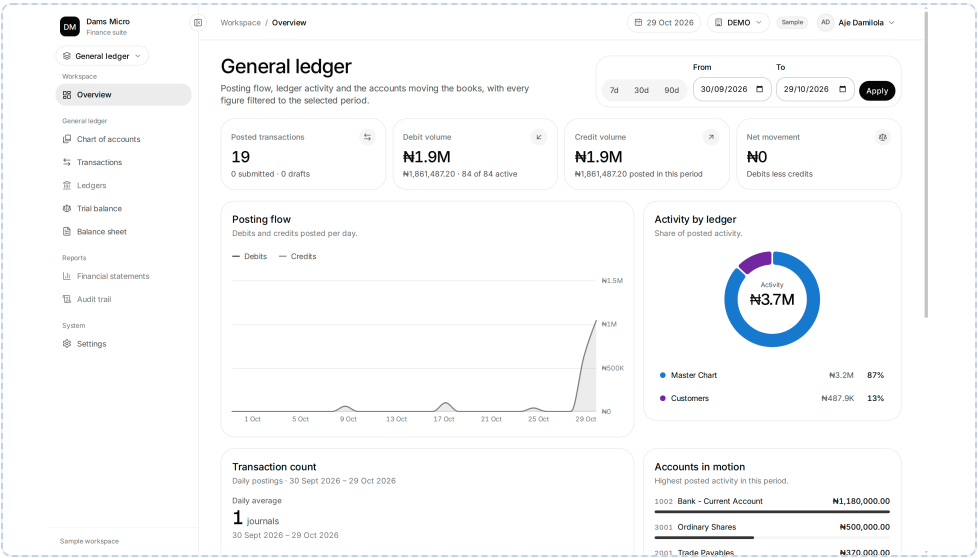
DEPARTMENT
[TO FILL: Department]

INSTITUTION
[TO FILL: Institution]

ORGANISATION ATTACHED
[TO FILL: Organisation]

SIWES PERIOD
[TO FILL: Start – End]

Built with Laravel 13 · PostgreSQL · React 19 and Inertia · Tailwind CSS 4



Hero screenshot: the general ledger dashboard.

One system. Two sides. Balanced books.

ABOUT THE ORGANISATION

A microfinance bank runs on trust in its books

Small loans and savings only work when the records behind them are exact.

Placement | [TO FILL: Start – End]

[TO FILL: Organisation name]

[TO FILL: What the organisation does, in one line.]

A microfinance bank serves people and small businesses that mainstream banks often do not reach.

I worked on the core system that turns deposits, loans and repayments into balanced journal entries.

Placement unit	My role	Placement focus
[TO FILL: Unit]	[TO FILL: Role]	**Core system design and build**

Organisation logo

`images/logo.png`

[TO FILL: add logo if supplied]

What I observed

Every product, every fee and every repayment eventually lands in one place: the books.

OBJECTIVES AND SCOPE

My scope was the core, not the edges

SIWES is the bridge between the classroom and the workplace. My assignment was to design and build the bank's core system.

Why SIWES

01 Apply software engineering to a live finance problem.

02 Learn how accounting controls shape software.

03 Deliver something the bank can use after SIWES.

Success meant a system the finance team could trust.

What I owned

- ✓ General ledger, chart of accounts and subledgers.
- ✓ Deposit products and the full loan lifecycle.
- ✓ End-of-day controls, tests and the user guide.

Success meant the books balanced after every action.

Out of scope: live interbank settlement. External transfers are simulated.

Simulated

THE PROBLEM

Manual books cannot prove that every naira balances

A microfinance bank needs three things from software: correct balances, a loan book that matches the ledger, and a day-end it can prove.

Spreadsheets don't enforce double entry

One mistyped cell breaks the books silently, and nothing flags it.

Control gap

Loan trackers live outside the ledger

Interest, fees and penalties drift away from the official books.

Reconciliation gap

Day-end is a manual routine

The date can move with no proof of what was checked first.

Audit gap

⚠ The risk is the same everywhere: the business cannot prove what it owns, owes or earned.

THE SOLUTION

One core keeps the books and the loan book in step

MFB Core System is a double-entry core: every deposit, loan and repayment posts to the general ledger.

General Ledger

Balanced journals, trial balance and balance sheet.

Chart of Accounts

Control accounts with customer subledgers underneath.

Deposits

Deposit products, interest accrual and ledger postings.

Loans

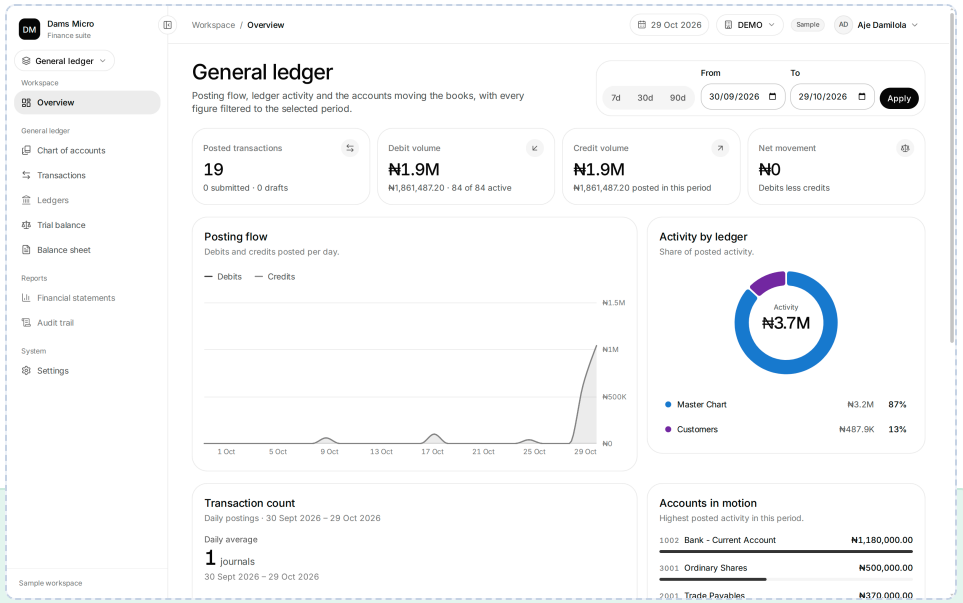
KYC, schedules, repayments and GL mapping.

Customers

KYC tiers, documents and verification status.

End-of-Day

One atomic Advance Day control with pre-checks.



No side ledgers

- ✓ Loan events post automatically to the ledger.
- ✓ Corrections are reversals, never silent edits.

SYSTEM ARCHITECTURE

Four layers, one rule: the engine owns every posting

The UI, the application and the accounting engine are separate on purpose.

USERS & ROLES
Who can see and do what

Admin · Accountant · Credit Manager · Auditor

APPLICATION
Server-rendered pages and actions

Laravel 13 · PHP 8.4 · React 19 · Inertia v3 · Tailwind 4

ACCOUNTING ENGINE
Every balance-changing rule

Journal posting · Balance resolution · Accruals & schedules · Operation → GL mapping · EOD snapshot

DATABASE
The durable record

PostgreSQL · Companies · Ledgers · Journals · Loans

↓↑ Requests flow down. Only the engine writes balances. The database never trusts the UI.

ACCOUNTING BACKBONE

Customer balances live under a control account

The chart holds control accounts. Customer detail lives in a subledger behind each control.

GENERAL LEDGER

Chart of Accounts

One chart per company, grouped into assets, liabilities, equity, revenue and expenses.

balances roll up into a summary account

CONTROL ACCOUNT

1100 Customer Control

One summary balance for every customer in the company.

detail lives below the control

SUBLEDGER

Customers Subledger

Individual customer balances, created when a customer is onboarded.

`CUST-001` `CUST-002` `CUST-003` `sample accounts`

Only leaf accounts accept postings. Control accounts never do.

General ledger
The complete set of accounts for one company.

Control account
A summary account whose balance equals its subledger total.

Subledger
The detail accounts behind a control account.

The rule
Customer balances sit in the subledger, not as individual chart accounts.

TRANSACTION FLOW

Every transaction becomes a balanced journal entry

A journal entry is one financial event written as debits and credits that must sum to zero.

01
Event
Customer deposits ₦50,000 at the counter.

→

02
Journal
Dr Cash on Hand · Cr Customer Deposit.

→

03
Ledger
Both entries post to their accounts.

→

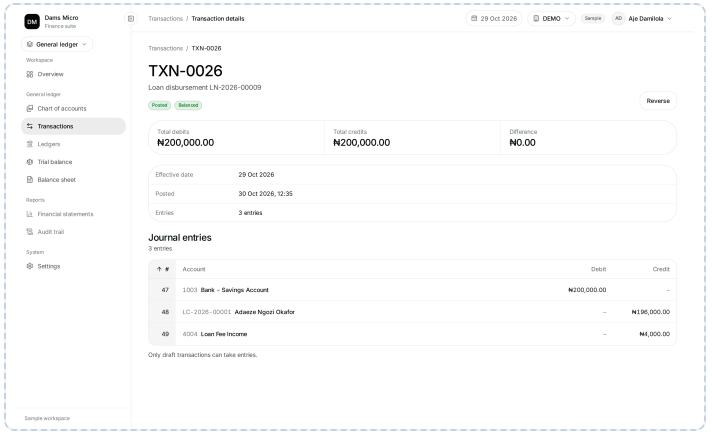
04
Balance
Opening balance plus today's movement.

WORKED EXAMPLE · DEPOSIT AT THE COUNTER

● Balanced

Account	Debit	Credit
Cash on Hand	₦50,000.00	–
Customer Deposit	–	₦50,000.00
Total	**₦50,000.00**	**₦50,000.00**

One event, two entries, one balance. Posted entries are final; a mistake is reversed, never edited.



The journal screen: entries, totals and status.

PRODUCTS

Deposits and loans meet in the same general ledger

Deposits earn interest over time. Loans earn interest before the cash arrives.

Deposit products

Interest accrual

- Interest accrues by business date.
- Accrual is recognised before cash moves.
- Every posting lands in the general ledger.

Deposit account statement

`screenshots/deposit-statement.png`

The statement: deposits, withdrawals and running balance.

Loans

Loan lifecycle

Book → Approve → Disburse → Repay → Close

Dr	→	Cr
Interest Receivable		**Interest Income**
asset · what the borrower owes		revenue · what the bank earned

WORKED EXAMPLE · ₦200,000 LOAN, 2% FEE

Balanced

Account	Debit	Credit
Bank · settlement	₦200,000.00	–
Customer Account	–	₦196,000.00
Fee Income	–	₦4,000.00
Total	**₦200,000.00**	**₦200,000.00**

Accrual recognises income before cash arrives. Both products share one ledger.

END OF DAY

Advance Day closes one date and opens the next

End-of-day (EOD) is the controlled close of one business date. Here it is one atomic action.

Pre-checks

• All must pass

The date does not move until the queue is clean.

- ✓ No draft journals
- ✓ No submitted journals
- ✓ Target date is later
- ✓ No other advance is running

Advance Day

One action moves the financial clock.

ADVANCE DAY

Lock · snapshot · advance · log

One database transaction. Two advances cannot interleave.

New business date

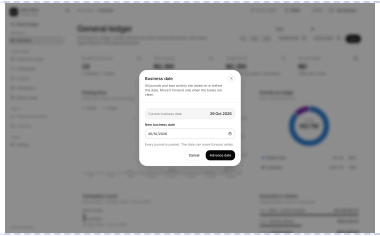
`Day N → Day N+1`

- **Close the old day**
  Closing balances are captured.
- **Carry the balance forward**
  Closing becomes opening.
- **Open the new day**
  New postings use the new date.

Sample: 09 Nov 2026 → 10 Nov 2026.

What is logged

Status `COMPLETED`	New date `10 Nov 2026`
Evidence `checks + snapshot`	On failure `reason captured`



The settings screen: checks first, then the date moves.

KEY DESIGN DECISIONS

Three decisions traded convenience for auditability

Each choice makes the common case fast, while keeping a path back to the truth.

DECISION 01

Store `balance_after` on every entry

WHY

Statements and audit reads stay fast without replaying history.

TRADE-OFF

Corrections need a rebuild path for the affected accounts.

DECISION 02

Cache a display balance, resolve the truth

WHY

Lists stay quick; opening plus movements prove the balance.

TRADE-OFF

The cache can drift; snapshots and rebuilds reconcile it.

DECISION 03

Simulated

Simulate external transfers

WHY

No live NIBSS or NIP rail is connected in this environment.

TRADE-OFF

No real interbank settlement yet; a production rail is required.

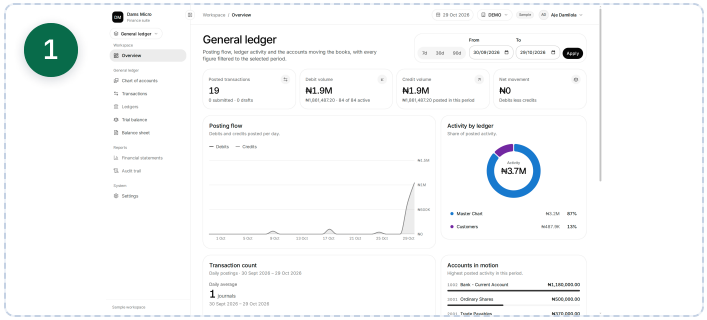
The pattern: optimise for speed, but never lose the path back to the ledger.

DEMO

Four screens show the core working

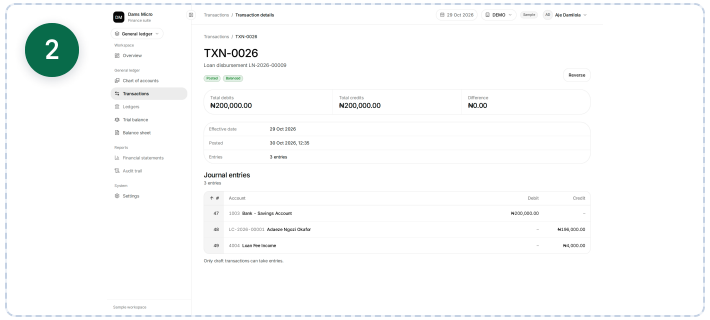
Screenshot frames are in place. The real captures will come from the running app with Playwright.

1



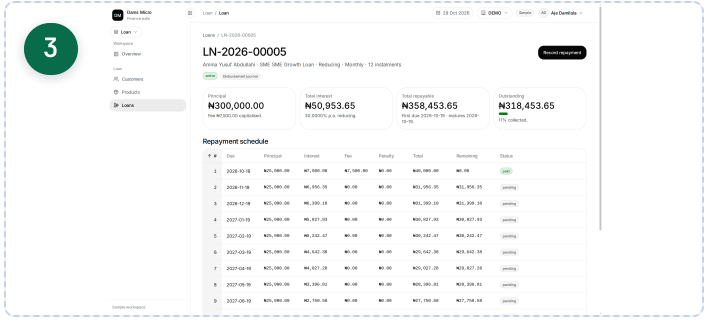
Postings, volumes and recent activity.

2



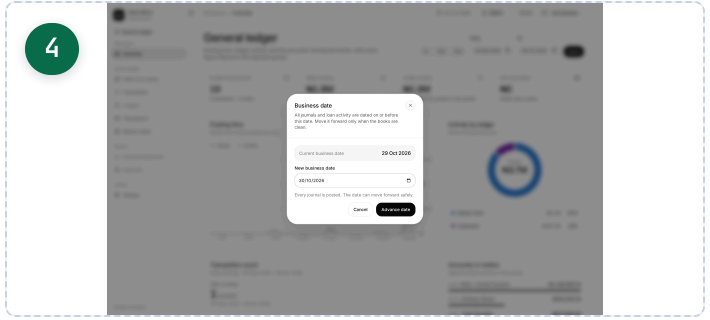
Balanced entries and their status.

3



Schedule, repayments and linked journal.

4



Pre-checks and the next business date.

1 Totals that balance
Debits equal credits on every posted journal.

2 Every entry linked
A repayment opens its journal in one click.

3 Schedule vs paid
Each instalment shows due, paid and outstanding.

4 Gate before the date moves
Advance Day refuses to run with unposted work.

Placeholders are shown until the Playwright capture is added to each file path.

CHALLENGES AND FIXES

Three hard problems, three deliberate fixes

These are the places where a simple implementation would have made the books lie.

Challenge

Running balances drift after corrections

A stored running balance can disagree with the ledger.

Fix

Rebuild from posted entries

`balance_after` is stored per entry; corrections are reversals, and a rebuild replays posted entries in date order.

Challenge

Customer detail would bloat the chart

Thousands of customer accounts would swamp the chart of accounts.

Fix

Control account plus subledger

Customer balances sit behind Customer Control; postings to control accounts are blocked.

Challenge

Day-end could close a half-finished day

A manual checklist can miss a draft or submitted journal.

Fix

One atomic Advance Day transaction

Lock, check the unposted queue, snapshot balances, advance and log, all or nothing.

SKILLS AND CONCLUSION

I can now build finance systems, not just screens

The placement changed how I think about correctness, controls and the people who use the software.

Technical

- ✓ Laravel, PostgreSQL, React and Inertia.
- ✓ Double-entry accounting in code.
- ✓ Feature tests for money paths and day-end.

Professional

- ✓ Turning finance rules into requirements.
- ✓ Writing a user guide for accountants.
- ✓ Explaining trade-offs to non-engineers.

What I would improve next

Live NIBSS / NIP rail

More automated reconciliation tests

RECOMMENDATION

Keep the business-date discipline: close the day before opening the next.

The measure is not the screens. It is books that balance.

SIWES · [TO FILL: START – END]

Thank you.

Questions?

[TO FILL: Full name] · [TO FILL: Matric no.]

[TO FILL: Email]

[TO FILL: Phone]

[TO FILL: Portfolio or LinkedIn]

Every naira, both sides.

MFB Core System · double-entry accounting and lending core.

Debits	₦50,000.00
Credits	₦50,000.00
Difference	₦0.00